



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Informatyki
Środowiska Udostępniania Usług
Grupa 4 - czwartek 13:15

MCP-PK

*Obserwowalność Kubernetes z Prometheusem kontrolowanym
przez serwer MCP*

*Kubernetes Observability with Prometheus controlled by MCP
server*

Autorzy: Dominik Drózdź, Szymon Miękina, Magdalena Pabisz, Marcin Walendzik

Kraków, 2026

Spis treści

1	Wprowadzenie	3
2	Podstawy teoretyczne i stos technologiczny	4
2.1	Kubernetes	4
2.2	Prometheus	4
2.3	MCP Server	5
2.4	Large Language Model	5
2.5	Grafana	5
3	Opis studium przypadku	6
3.1	Aplikacja testowa: Microservices-demo	6
3.1.1	Architektura i zasada działania	6
3.1.2	Uzasadnienie wyboru	7
3.2	Opis przypadku użycia	7
4	Wysokopoziomowy opis architektury	8
5	Szczegółowy opis architektury	9
5.1	Metryki i endpointy aplikacji Microservices Demo	9
6	Konfiguracja środowiska	12
6.1	Utworzenie klastra EKS	12
6.2	Konfiguracja dostępu do klastra	12
6.3	Weryfikacja konfiguracji środowiska	12
7	Sposób instalacji	14
7.1	Kubernetes i uruchomienie aplikacji	14
7.1.1	Menadżer pakietów Kubernetes HELM	15
7.1.2	Monitoring i wizualizacja	15
7.1.3	Serwer MCP	16
8	Wdrożenie wersji demonstracyjnej	17
9	Opis demonstracji	18
10	Podsumowanie i wnioski	19
	Spis rysunków	20
	Spis tabel	21

Rozdział 1

Wprowadzenie

Obserwowalność (observability) jest kluczowym elementem nowoczesnych systemów cloud-native. Umożliwia pomiar, analizę oraz zrozumienie wewnętrznego stanu systemu na podstawie jego zewnętrznych wyników. Opiera się na trzech głównych typach danych: metrykach, które pozwalają monitorować ogólną efektywność systemu, logach, które opisują zdarzenia zachodzące w systemie, oraz śladach, które umożliwiają śledzenie przepływu operacji.

Współczesne aplikacje coraz częściej składają się z niezależnych mikroserwisów i działają w środowiskach kontenerowych, takich jak Kubernetes. W systemach rozproszonych niemożliwe jest ręczne śledzenie zależności pomiędzy usługami i przewidywanie potencjalnych awarii. Z tego względu niezbędne jest zastosowanie narzędzi wspierających obserwowalność. Kubernetes nie posiada wbudowanych mechanizmów monitorowania, dlatego integruje się go z zewnętrznymi narzędziami, takimi jak Prometheus. Dzięki obserwowalności można zrozumieć zależności pomiędzy poszczególnymi usługami, zlokalizować nieefektywne fragmenty kodu czy wolne zapytania do bazy danych. W konsekwencji odpowiednie monitorowanie pozwala zapewnić niezawodny, wydajny i skalowalny system.

Celem projektu jest przygotowanie rozwiązania do obserwowalności aplikacji uruchomionej w środowisku Kubernetes z wykorzystaniem Prometheusa. Dodatkowo infrastruktura odpowiedzialna za obserwowalność jest rozszerzona o serwer MCP (Model Context Protocol), który pozwala na wykorzystanie dużego modelu językowego. Dzięki temu możliwe jest zarządzanie konfiguracją Prometheusa za pomocą poleceń w języku naturalnym. Serwer MCP pełni rolę warstwy pośredniczącej, która przekształca język naturalny na operacje konfiguracyjne wykonywane w Prometheusie. Takie podejście pozwala częściowo zautomatyzować zarządzanie infrastrukturą monitorującą oraz zwiększyć elastyczność i wygodę jej obsługi.

Rozdział 2

Podstawy teoretyczne i stos technologiczny

Wybór technologii do tworzenia projektu został częściowo podyktowany sugestiami zawartymi w dokumentach z zajęć laboratoryjnych przedmiotu Środowiska Udostępniania Usług, jak i również powszechnością konkretnych rozwiązań w przypadku danej funkcjonalności. Większość z poniżej opisanych narzędzi jest rozwijana jako technologie typu open-source, co zapewnia często większą swobodę przy korzystaniu z produktu.

2.1. Kubernetes

Kubernetes jest narzędziem stworzonym do orkiestracji kontenerów. Stanowi obecnie najpowszechniejszą technologię z tego zakresu. Jej produkcja została rozpoczęta przez Google, a obecnie jest rozwijana jako produkt otwartoźródłowy. Kubernetes opiera swoje działanie o kontrolę nad aplikacjami zamkniętymi w kontenerach. Technologia ta pozwala na dynamiczne zarządzanie usługami zapewniając łatwą skalowalność, kluczową obecnie w wielkich środowiskach usługowych. System dostosowuje automatycznie ilość potrzebnych instancji kontenerów w oparciu o zdefiniowane przez deweloperów reguły oraz obciążenie. W kontekście naszego projektu, warto wspomnieć również, że Kubernetes wystawia endpointy, które służą warstwie observability do czytania danych metrycznych pomagających określić stan serwisów oraz zasobów sprzętowych.

2.2. Prometheus

Prometheus to technologia open-source służąca do monitorowania stanów zasobów w złożonych aplikacjach i systemach o wielu usługach. Zarządza on danymi wystawionymi przez aplikację, agregując je w bazie danych typu time-series database. W stosunku do aplikacji działa jako klient, który aktywnie pobiera wyznaczone dane dostępne na endpointach HTTP. Skupia się on na danych numerycznych z zakresu m.in. zużycia zasobów pamięci operacyjnej oraz masowej, ilości wystąpień wyjątków, czasu odpowiedzi aplikacji na zapytania. Dane te są dostępne następnie za pomocą wbudowanego języka zapytań PromQL. Narzędzie to udostępnia również mechanizm powiadamiania wskazanych użytkowników nt. awarii, bądź nieefektywnego zużycia poszczególnych zasobów. W kontekście wielousługowych aplikacji Prometheus jest obecnie pomocnym i skutecznym narzędziem pozwalającym na ciągłe monitorowanie stanu aplikacji, dostosowywanie zasobów pod wskazany cel, będąc jednocześnie dostępnym w formie wolnego oprogramowania.

2.3. MCP Server

MCP Server jest warstwą stanowiącą pomost między instrukcją w języku naturalnym a konfiguracją w systemie. Działa w oparciu o otwartoźródłowy protokół internetowy MCP (*Model Context Protocol*). Udostępnia on warstwę komunikacyjną dla modelu, dzięki której może on dynamicznie korzystać z zasobów i funkcji serwera.

2.4. Large Language Model

Duży model językowy jest elementem najistotniejszym z perspektywy instrukcji wejściowych. Od strony technicznej jest to specjalnie wytrenowana sieć neuronowa zdolna do generowania tekstu. W przypadku opisywanego projektu powinna stanowić pośrednika, który będzie umiał przetłumaczyć instrukcje użytkownika w języku naturalnym na odpowiednie zapytania techniczne i operacje konfiguracyjne. Jego wyjście kierowane do serwera MCP sprawi, że użytkownik będzie mógł aktywnie konfigurować element observability z pozycji instrukcji sformułowanych "po ludzku".

2.5. Grafana

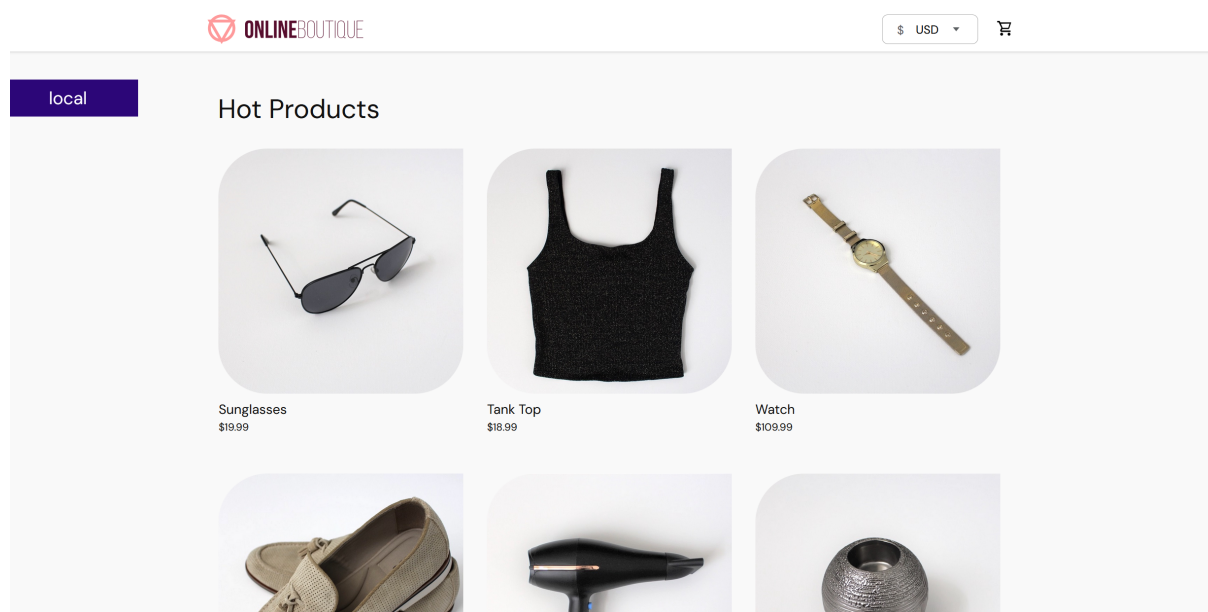
Grafana jest elementem służącym do wizualizacji danych przechowywanych w bazie danych Prometheusa. Jej przewagą jest większa możliwość w kwestii wizualizacji. Dzięki niej można utrzymywać kompozycje różnych wykresów zależnie od celu. Grafana, podobnie jak poprzednie technologie, jest również dostępna jako technologia open-source.

Rozdział 3

Opis studium przypadku

3.1. Aplikacja testowa: Microservices-demo

W projekcie jako obiekt monitorowania wykorzystano aplikację **Google Microservices Demo** (znaną również pod nazwą *Online Boutique*). Jest to otwartoźródłowa aplikacja demonstracyjna, która symuluje działanie sklepu internetowego. Składa się ona z 11 mikroservisów napisanych w różnych językach programowania (m.in. Go, Python, Java, C#, Node.js), które komunikują się ze sobą za pomocą protokołu gRPC.



Rysunek 3.1: Aplikacja Online Boutique

3.1.1. Architektura i zasada działania

Aplikacja została zaprojektowana w sposób natywny dla chmury (*cloud-native*), co oznacza, że każdy jej komponent pełni ściśle określoną rolę:

- **Frontend:** Serwer WWW napisany w Go, który serwuje interfejs użytkownika.
- **Product Catalog Service:** Dostarcza informacje o produktach z plików JSON.

- **Cart Service:** Zarządza koszykami użytkowników, wykorzystując bazę danych Redis do przechowywania danych tymczasowych.
- **Currency Service:** Odpowiada za przeliczanie walut.
- **Payment Service:** Obsługuje procesy płatności (symulacja).
- **Shipping Service:** Oblicza koszty dostawy.
- **Ad Service:** Serwuje reklamy kontekstowe.

Każdy z serwisów jest odizolowanym kontenerem, co pozwala na niezależne skalowanie komponentów wewnątrz klastra Kubernetes.

3.1.2. Uzasadnienie wyboru

Wybór tej konkretnej aplikacji do projektu obserwowalności z wykorzystaniem serwera MCP i Prometheusa jest podyktowany kilkoma kluczowymi czynnikami:

1. **Różnorodność technologiczna:** Dzięki zastosowaniu wielu języków i frameworków, aplikacja stanowi idealne środowisko do testowania uniwersalności narzędzi monitorujących.
2. **Złożoność ruchu sieciowego:** Komunikacja gRPC pomiędzy wieloma ogniwami systemu pozwala na generowanie realistycznych metryk dotyczących opóźnień (latency), liczby zapytań oraz błędów, co jest kluczowe dla demonstracji możliwości Prometheusa.
3. **Gotowość do wdrożenia na Kubernetes:** Aplikacja posiada natywne manifesty YAML oraz wsparcie dla Helm, co znacząco ułatwia jej szybką orkiestrację i integrację z infrastrukturą monitorującą.
4. **Realizm przypadku użycia:** Symulacja sklepu internetowego pozwala na łatwe definiowanie reguł biznesowych i alertów (np. powiadomienie o błędzie w procesie płatności lub zbyt wolnym ładowaniu katalogu produktów), które mogą być następnie modyfikowane za pomocą poleceń w języku naturalnym przez serwer MCP.

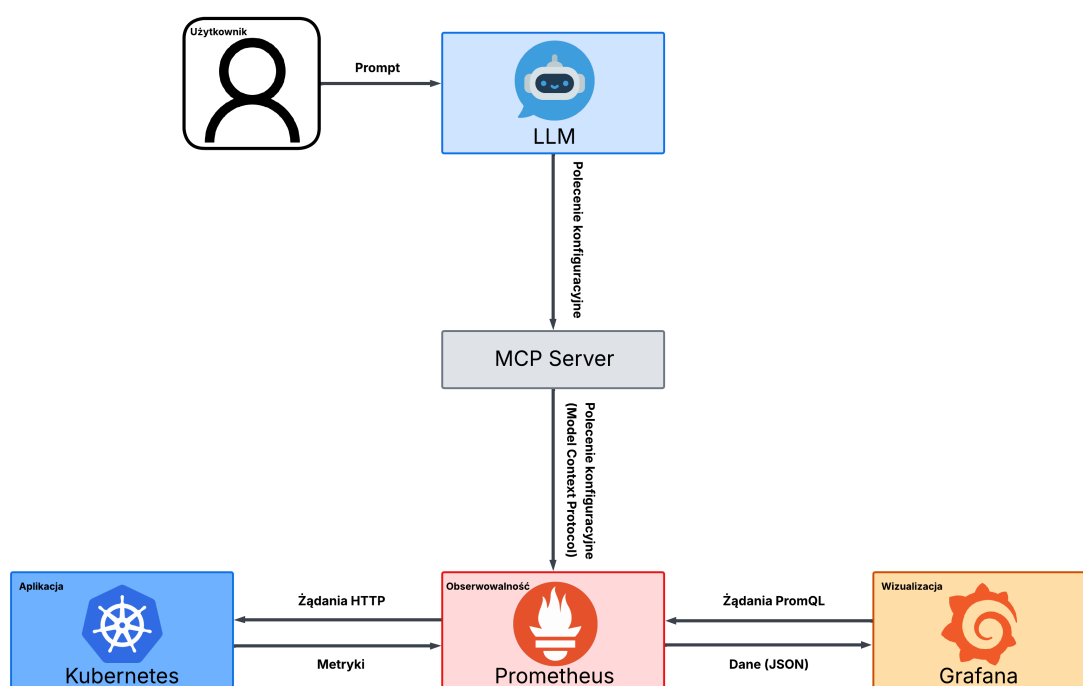
3.2. Opis przypadku użycia

Poniżej opisano kolejne operacje, które zostaną przeprowadzone w celu demonstracji przypadku użycia.

1. Uruchomienie klastra **Kubernetesa** z wybraną aplikacją oraz wdrożenie narzędzia monitorującego **Prometheus** wraz z podstawową konfiguracją zbierającą dane dotyczące zużycia CPU, a także integracja **serwera MCP** oraz systemu wizualizacji **Grafana** z Prometheusem.
2. Wydanie poleceń w języku naturalnym przy użyciu dużego modelu językowego, w wyniku których dokonane zostaną następujące zmiany w konfiguracji Prometheusa:
 - zmniejszenie interwału zbierania metryk,
 - dodanie reguły alertowania o wysokim zużyciu procesora,
 - rozszerzenie monitorowania o metrykę zużycia pamięci (memory usage).
3. Zaobserwowanie wprowadzonych zmian w konfiguracji w Grafanie.

Rozdział 4

Wysokopoziomowy opis architektury



Rysunek 4.1: Architektura wysokopoziomowa

Rozdział 5

Szczegółowy opis architektury

5.1. Metryki i endpointy aplikacji Microservices Demo

Aplikacja *Google Microservices Demo* została zaprojektowana w sposób umożliwiający łatwą integrację z narzędziami obserwowalności, takimi jak Prometheus. Poszczególne mikroserwisy udostępniają endpointy HTTP wykorzystywane do monitorowania stanu aplikacji oraz zbierania danych metrycznych.

W środowisku Kubernetes Prometheus wykorzystuje mechanizm *scrapingu*, polegający na okresowym odpytywaniu wskazanych endpointów HTTP i pobieraniu danych zapisanych w formacie tekstowym zgodnym ze standardem Prometheusa.

Endpointy monitorujące

W przypadku aplikacji Online Boutique najważniejsze endpointy obejmują:

Każdy endpoint `/metrics` zwraca dane w postaci tekstowej. Poniżej przedstawiono przykładowy fragment odpowiedzi:

```
1 http_requests_total{method="GET",status="200"} 1523
2 http_request_duration_seconds_sum 12.51
3 process_cpu_seconds_total 35.2
4 process_resident_memory_bytes 1.24e+08
```

Listing 5.1: Przykładowe dane zwracane przez endpoint `/metrics`

Metryki aplikacyjne

W projekcie wykorzystano zarówno metryki infrastrukturalne, jak i aplikacyjne. Najważniejsze z nich przedstawiono w tabeli 5.2.

Metryki infrastrukturalne Kubernetes

Oprócz danych aplikacyjnych monitorowane są również zasoby klastra Kubernetes. W tym celu wykorzystano komponenty:

- **Node Exporter** – dostarcza metryki systemowe hostów,
- **Kube State Metrics** – udostępnia informacje o stanie obiektów Kubernetes,
- **cAdvisor** – monitoruje zużycie zasobów przez kontenery.

Tabela 5.1: Przykładowe endpointy aplikacji wykorzystywane przez Prometheusa

Serwis	Endpoint	Opis
Frontend	/metrics	Udostępnia metryki HTTP oraz statystyki ruchu użytkowników.
Cart Service	/metrics	Dane dotyczące operacji koszyka oraz komunikacji z Redisem.
Product Catalog Service	/metrics	Metryki związane z czasem odpowiedzi i odczytem katalogu produktów.
Payment Service	/metrics	Statystyki przetwarzania płatności oraz liczby błędów.
Shipping Service	/metrics	Dane dotyczące obliczania kosztów dostawy.
Kubernetes Node Exporter	/metrics	Metryki systemowe węzłów klastra Kubernetes.
Kube State Metrics	/metrics	Informacje o stanie obiektów Kubernetes.

Tabela 5.2: Najważniejsze metryki monitorowane w projekcie

Metryka	Opis	Typ
http_requests_total	Całkowita liczba żądań HTTP obsługiwanych przez serwis.	Counter
http_request_duration_seconds	Czas odpowiedzi endpointów HTTP.	Histogram
process_cpu_seconds_total	Łączny czas wykorzystania CPU przez proces.	Counter
process_resident_memory_bytes	Zużycie pamięci RAM przez proces.	Gauge
container_cpu_usage_seconds_total	Zużycie CPU kontenera w Kubernetes.	Counter
container_memory_usage_bytes	Aktualne zużycie pamięci przez kontener.	Gauge
grpc_server_handled_total	Liczba obsługiwanych wywołań gRPC.	Counter
grpc_server_handling_seconds	Czas obsługi żądań gRPC.	Histogram

Przykładowe metryki infrastrukturalne obejmują:

- wykorzystanie CPU i pamięci przez kontenery,
- liczbę restartów podów,
- stan replikacji Deploymentów,
- wykorzystanie przestrzeni dyskowej,
- ruch sieciowy pomiędzy mikroserwisami.

Konfiguracja scrapingu w Prometheusie

Prometheus został skonfigurowany do okresowego pobierania danych metrycznych z endpointów aplikacji. Przykładowa konfiguracja wygląda następująco:

```
1 scrape_configs:
2   - job_name: 'frontend'
3     scrape_interval: 5s
4     metrics_path: /metrics
5
6     kubernetes_sd_configs:
7       - role: pod
8
9     relabel_configs:
10      - source_labels: [__meta_kubernetes_pod_label_app]
11        action: keep
12        regex: frontend
```

Listing 5.2: Przykładowa konfiguracja scrape job w Prometheusie

W przedstawionej konfiguracji Prometheus pobiera dane z endpointu `/metrics` co 5 sekund dla wszystkich podów oznaczonych etykietą `frontend`.

Wykorzystanie metryk w Grafanie

Zebrane metryki zostały następnie wykorzystane do budowy dashboardów w Grafanie. Dashboardy umożliwiają:

- obserwację czasu odpowiedzi poszczególnych mikroserwisów,
- monitorowanie obciążenia CPU i pamięci,
- analizę liczby błędów HTTP oraz gRPC,
- identyfikację przeciążonych komponentów aplikacji,
- śledzenie komunikacji pomiędzy mikroserwisami.

Dzięki integracji Prometheusa z Grafaną możliwe jest zarówno monitorowanie infrastruktury Kubernetes, jak i analiza działania samej aplikacji biznesowej.

Rozdział 6

Konfiguracja środowiska

Projekt został wykonany w oparciu o chmurę publiczną **AWS** oraz udostępniony w ramach zajęć kurs AWS Academy Learner Lab. Środowisko zostało zbudowane z wykorzystaniem usługi **Amazon Elastic Kubernetes Service (EKS)**. W ramach konfiguracji utworzono dedykowany klaster Kubernetes przeznaczony do realizacji projektu.

6.1. Utworzenie klastra EKS

W pierwszym kroku utworzono klaster Kubernetes o nazwie `suu-project`. Klaster został skonfigurowany w regionie AWS `us-east-1`. W ramach infrastruktury obliczeniowej dodano grupę węzłów (node group) składającą się z instancji EC2 typu `t3.large`.

6.2. Konfiguracja dostępu do klastra

W celu umożliwienia komunikacji z klastrem skonfigurowano lokalne środowisko użytkownika poprzez plik:

```
~/.aws/credentials
```

zawierający dane uwierzytelniające AWS CLI.

Następnie, za pomocą AWS CLI, zweryfikowano status klastra:

```
aws eks describe-cluster --region us-east-1 --name suu-project --query cluster.status
```

Skonfigurowano również dedykowane narzędzie `kubectl` do komunikacji z klastrem EKS:

```
aws eks --region us-east-1 update-kubeconfig --name suu-project
```

6.3. Weryfikacja konfiguracji środowiska

Po wykonaniu powyższych kroków zweryfikowano poprawność konfiguracji środowiska Kubernetes.

Sprawdzono dostępność węzłów klastra za pomocą polecenia:

```
kubectl get nodes
```

Dodatkowo zweryfikowano stan komponentów systemowych Kubernetesa działających w namespace kube-system:

```
kubectl --namespace kube-system get pods
```

Rozdział 7

Sposób instalacji

W celu stworzenia założonego projektu dołączono kilka narzędzi zajmujących się jego wyszczególnionymi częściami. Wszystkie użyte narzędzia były darmowe, a zdecydowana większość z nich znajduje się aktualnie pod opieką CNCF (*ang. Cloud Native Computing Foundation*).

7.1. Kubernetes i uruchomienie aplikacji

Tak więc do postawienia wybranej aplikacji mikroserwisowej w posiadanym środowisku użyto Kubernetesa. W celu jego konfiguracji użyto dedykowany dla tej usługi interfejs wiersza poleceń - `kubectl`.

Instalacja tego narzędzia może przebiegać w różny sposób. Najprostsze sposoby to pobranie binarnego pliku instalacyjnego dostępnego na oficjalnej stronie (<https://kubernetes.io/releases/download/#binaries>).

Można również przeprowadzić instalację przy pomocy menadżera pakietów systemu operacyjnego. W przypadku systemu Windows, który był domyślnym wyborem dla członków zespołu, pobranie i konfiguracja narzędzia odbywała się poprzez komendę wiersza poleceń:

```
winget install Kubernetes.kubectl
```

Instalacja przez menadżer pakietów pozwala na automatyczne zarządzanie instalacją przez algorytm. Dodatkowo, sposób ten zapewnia poprawne dodanie nowego narzędzia do zmiennych środowiskowych. W przypadku instalacji poprzez plik binarny, konieczne jest ręczne dodanie folderu z plikami do zmiennych systemowych.

Po poprawnie przeprowadzonej instalacji, w systemie po wpisaniu poniższej komendy użytkownik powinien otrzymać aktualną wersję narzędzia:

```
User> kubectl version --client  
Client Version: v1.32.2  
Kustomize Version: v5.5.0
```

Przy założeniu sukcesu, następnym krokiem jest uruchomienie aplikacji na klastrze obliczeniowym z usługi AWS. Po komendzie apply klaster uruchamia aplikację bazującą na opisie z pliku google-microservices-manifests.yaml:

```
kubectl apply -f google-microservices-manifests.yaml
```

7.1.1. Menadżer pakietów Kubernetes HELM

Posiadając uruchomioną aplikację, pozostałe czynności poszerzające jej funkcjonalność do kształtu zamierzonego projektu polegają na definiowaniu manifestów kubernetes i wdrażanie ich poprzez wiersz poleceń. Proces ten ręczny jest długi i monotony, ale został on już zautomatyzowany przez specjalne narzędzia.

Oprogramowanie HELM jest menadżerem pakietów do aplikacji bazujących na systemie kubernetes, który automatycznie generuje podstawową konfigurację z pozycji wiersza poleceń. Narzędzie to pozwala na proste i szybkie dołączenie do projektu wymaganych z założeń modułów.

Narzędzie również wymaga zainstalowania w analogiczny sposób, co narzędzie kubectl. Na oficjalnym repozytorium dystrybuowane są pliki instalacyjne (<https://github.com/helm/helm/releases>), ale również można skorzystać z narzędzi systemowych:

```
winget install Helm.Helm
```

7.1.2. Monitoring i wizualizacja

Korzystając z HELM dołączenie kolejnych modułów projektu było bardzo proste i szybkie. Dodanie systemu monitorowania jakim jest Prometheus wraz z Grafaną jest ustawiane poprzez szereg komend:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
helm repo update
```

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.enabled=true
```

Pomyślne przeprowadzenie tego kroku będzie dawało efekty w postaci nowych pod'ów oraz serwisów z przestrzeni nazw monitoring. Aby przetestować działanie grafany ustawiono przekazywanie portów na port lokalny, a następnie zalogowanie się do grafany poprzez otrzymanie odpowiednich kluczy dostępowych.

```
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
```

7.1.3. Serwer MCP

Ostatnim krokiem zostało dodanie elementu umożliwiającego komunikację modelu językowego z systemem monitorowania Prometheus - dodanie MCP server. Również dzięki HELM proces ten można wykonać bardzo szybko:

```
helm install prometheus-mcp-server oci://ghcr.io/pablito0/charts/prometheus-mcp-server -
```

```
kubectl port-forward svc/prometheus-mcp-server 8080:8080 -n default
```

Utworzony serwer można przetestować poprzez sprawdzenie sprawności poszczególnych endpoint'ów. Do sprawdzenia statusu serwera służy m.in. endpoint /health.

Powyżej przedstawiona instalacja pozwala na otrzymanie podstawowej wersji serwera kompletnej we wszystkie istotne dla kształtu projektu komponenty.

Rozdział 8

Wdrożenie wersji demonstracyjnej

Rozdział 9

Opis demonstracji

Rozdział 10

Podsumowanie i wnioski

Spis rysunków

3.1	Aplikacja Online Boutique	6
4.1	Architektura wysokopoziomowa	8

Spis tabel

5.1	Przykładowe endpointy aplikacji wykorzystywane przez Prometheusa	10
5.2	Najważniejsze metryki monitorowane w projekcie	10